

TRABAJO PRÁCTICO INTEGRADOR FOOD STORE – SISTEMA DE GESTIÓN DE PEDIDOS DE COMIDA

UNIVERSIDAD TECNOLÓGICA NACIONAL – FRM ANEXO RIVADAVIA

Tecnicatura Universitaria en Programación

Materia: Programación 2

Alumnos: Arancibia Exequiel

Romero Joaquin

Comisión: 5

Fecha: 21 de Junio 2026

Índice

1. Introducción
2. Marco Teórico
3. Análisis del Problema
4. Arquitectura del Sistema
5. Modelo UML
6. Implementación
7. Capturas del Sistema
8. Validaciones y Excepciones
9. Dificultades y Soluciones
10. Conclusiones
11. Bibliografía
12. Enlaces del Proyecto

Introduccion

El presente Trabajo Práctico Integrador consiste en el desarrollo de una aplicación de consola denominada Food Store, orientada a la gestión de productos, categorías, usuarios y pedidos de un negocio gastronómico.

El sistema fue desarrollado utilizando Java 21 y Programación Orientada a Objetos, aplicando conceptos de herencia, encapsulamiento, polimorfismo, interfaces, manejo de excepciones y almacenamiento en memoria mediante colecciones.

La aplicación permite realizar operaciones CRUD completas desde un menú de consola, respetando las reglas de negocio definidas por la cátedra.

• Marco Teórico

Java es un lenguaje orientado a objetos, multiplataforma y ejecutado mediante la JVM.

Durante el desarrollo se aplicaron conceptos de Programación Orientada a Objetos como clases, objetos, herencia, encapsulamiento, abstracción y polimorfismo.

Para el almacenamiento temporal de la información se utilizaron colecciones dinámicas, principalmente ArrayList.

También se implementó una interfaz denominada Calculable para estandarizar el comportamiento relacionado con el cálculo de totales en los pedidos.

Finalmente, se aplicó manejo de excepciones mediante bloques try-catch y excepciones personalizadas para controlar errores y evitar cierres inesperados de la aplicación.

- **Análisis del Problema**

El objetivo del sistema es permitir la administración de categorías, productos, usuarios y pedidos mediante una aplicación de consola.

La solución desarrollada permite realizar operaciones de alta, baja lógica, modificación y consulta de información de manera organizada y segura.

- **Arquitectura del Sistema**

El proyecto fue organizado utilizando una estructura modular basada en paquetes.

Main: punto de entrada de la aplicación.

Services: lógica de negocio.

Entities: entidades del dominio.

Enums: enumeraciones utilizadas por el sistema.

Exceptions: excepciones personalizadas.

Esta organización facilita el mantenimiento, reutilización y escalabilidad del código.

- **Modelo UML**

El sistema se encuentra basado en el modelo UML proporcionado por la cátedra.

Todas las entidades heredan de la clase abstracta Base, la cual contiene los atributos comunes del sistema.

Las entidades principales son:

- Categoría
- Producto
- Usuario
- Pedido
- DetallePedido

Además se utilizan los enums Rol, Estado y FormaPago.

- **Implementación**

Se implementó herencia mediante la clase Base.

La clase Pedido implementa la interfaz Calculable para realizar el cálculo del total de la compra.

La información se almacena en memoria utilizando colecciones dinámicas.

Las eliminaciones se realizan mediante baja lógica utilizando el atributo eliminado.

- **Capturas del Sistema**

- Menú principal

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Gestión de Categorías  
2. Gestión de Productos  
3. Gestión de Usuarios  
4. Gestión de Pedidos  
0. Salir del Sistema  
Seleccione una opción:
```

- Gestión de categorías

```
--- MENÚ CATEGORÍAS ---  
1. Listar categorías  
2. Crear categoría  
3. Editar categoría  
4. Eliminar categoría  
0. Volver al menú principal  
Seleccione:
```

- Gestión de productos

```
--- MENÚ PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver al menú principal
Seleccione:
```

- Gestión de usuarios

```
--- MENÚ USUARIOS ---
1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al menú principal
Seleccione:
```

- Gestión de pedidos

```
--- MENÚ PEDIDOS ---
1. Listar pedidos
2. Crear nuevo pedido (Cargar Carrito)
3. Cambiar estado/forma de pago de un pedido
4. Eliminar pedido
0. Volver al menú principal
Seleccione:
```

- Listados

```
Seleccione: 1
Categoría [ID=1, Nombre=Hamburguesas, Descripción=Variedad de hamburguesas caseras]
Categoría [ID=2, Nombre=Bebidas, Descripción=Gaseosas, aguas y cervezas]
```

```
Seleccione: 1
Producto [ID=1, Nombre=Hamburguesa Simple, Precio=$4500.0, Stock=20, Cat=Hamburguesas, Disponible=true]
Producto [ID=2, Nombre=Hamburguesa Completa, Precio=$5800.0, Stock=15, Cat=Hamburguesas, Disponible=true]
Producto [ID=3, Nombre=Gaseosa Cola 500ml, Precio=$1500.0, Stock=50, Cat=Bebidas, Disponible=true]
```

```
Usuario [ID=1, Nombre=Juan Pérez, Email=juan@mail.com, Rol=USUARIO]
Usuario [ID=2, Nombre=Admin Coop, Email=admin@foodstore.com, Rol=ADMIN]
```

- **Validaciones y Excepciones**

Validaciones implementadas:

- Mail único para usuarios.
- Precio mayor o igual a cero.
- Stock mayor o igual a cero.
- Categorías válidas.
- Usuarios válidos.
- Cantidades mayores a cero.

Excepciones utilizadas:

- `EntidadNoEncontradaException`
- `StockInvalidoException`

Estas excepciones permiten controlar errores de forma segura.

- **Dificultades y Soluciones**

Durante el desarrollo se presentaron desafíos relacionados con la implementación de relaciones entre entidades, validaciones y manejo de colecciones.

La solución consistió en separar responsabilidades mediante servicios específicos y aplicar buenas prácticas de Programación Orientada a Objetos.

- **Conclusiones**

Este proyecto permitió aplicar de manera práctica los conceptos fundamentales de Programación Orientada a Objetos desarrollados durante la cursada.

Se logró implementar una aplicación funcional utilizando herencia, interfaces, excepciones y colecciones, cumpliendo con los requisitos planteados por la cátedra.

11. Bibliografía

- Documentación oficial de Java.
- Oracle Java Documentation.
- Material de Programación 2 UTN.

- Java Collections Framework.
- Java Exception Handling.

12. Enlaces del Proyecto

Repositorio GitHub:

<https://github.com/joakkoromero/FoodStore>

Video demostrativo:

<https://drive.google.com/file/d/1zcf2zrXzpz132WIKMuDc5lI8aS3EjXBO/view?usp=s>
haring